UNIT -4 ADA

NP-Hard and NP-Complete

This is basically a research topic. All the algorithms we have already learnt. some take polynomial time.

NP Hard - Satisfiability — 2^n

Polynomial Time	Exponential Time
Linear Search — n	0/1 knapsack — 2^n
Binary search — $\log n$	Traveling SP — 2^n
Insertion sort — n^2	Sum of subsets — 2^n
Merge sort — $n \log n$	Graph Coloring — 2^n
Matrix Multiplication - n^3	Hamiltonian cycle — 2^n

Like searching algorithms. Problem is searching and algorithms are linear search, Binary search. like these 0/1 knapsack takes 2^n time. Here we have categorize the algorithms into 2. Scope for research here is see for searching we have linear search takes n time and Binary search which is faster than that takes $\log n$ time. we are in search of algorithms which is much faster than these. which takes $O(1)$ time. Similarly merge sort takes $n \log n$ time and for sorting also we are searching for algo better than this.

So this is a research area. Exponential time is much bigger than polynomial time. Even n^{10} is smaller than 2^n for some large value of n. So these are very time consuming algorithms so we want polynomial time algo for them.

There is no solution found till yet for solving Exponential time problems to be solved in polynomial time. so there is a framework made on exponential time problems. That framework / guidelines is NP hard or NP-Complete.

Points to be Remembered

(1) When we are unable to solve exponential time problems in polynomial time then at least we try to show the relationship similarities b/w them so that if one problem is solved others will also be solved. try to relate the problem.

(2) When we are unable to write deterministic algos for Exponential time problems. write Non Deterministic algorithms.

The algorithms we write are deterministic i.e each & every statement we know their working clearly. we can write non deterministic algorithms also i.e we don't know how they will work. write steps which are deterministic and leave blank for statements which are non deterministic. If we are trying to write 0/1 knapsack problems i.e polynomial time algorithm then we may be knowing some of the statements & some not. By writing non deterministic algorithms we can preserve our research work so in future if some body else work on the same problem he can make non deterministic part as deterministic.

Nondeterministic Algo Example

```
Algorithm NSearch(A, n, key)        // Searches a key from an array A of size n
{
    j = Choice();                   -| // Choice is non-determ
    if (key == A[j])                // it is giving index of key element
                                    // if key element is present at
                                    // j, search successful
    {
        write(j);
        Success();                  -| // non determ stmt
    }
    write(0);
    Failure();                      -| // non-deterministic stmt
}
```

O(1)

It seems that the algo takes O(1) time (constant time). We assume all non determ stmt takes 1 unit of time. Preserving the logic so in future it may become deterministic. Tomorrow it will be logical. We write non-deterministic algo for exponential time algo in polynomial time.

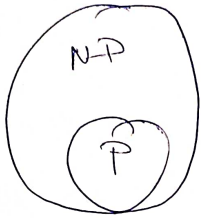
P — Set of those deterministic algorithms which are taking polynomial time.

Eg: Linear, Binary search, MCST, Huffman coding.

NP Class — Non deterministic algorithms takes polynomial time for which we will try to write algos which completes in polynomial time.

Eg merge sort is non deterministic in the past. we were not knowing that there is a possible algorithm

that takes nlogn time. we were working on Bubble sort which is taking n^2 time. So that algo was non-deterministic and became deterministic now. so when we came to know the algorithm it becomes deterministic. So we say that deterministic is a subset of nondeterministic or P is a subset of NP



(2) For relating Exponential time algos together we need some problem as a Base Problem. Base Problem is Satisfiability Problem.

CNF - Satisfiability

Propositional calculus formula $x_i = \{x_1, x_2, x_3\}$ — Boolean variable

Example formula $CNF = (x_1 \vee \bar{x}_2 \vee x_3) \wedge (\bar{x}_1 \vee x_2 \vee \bar{x}_3)$

C_1 C_2

Satisfiability problem is to find out for what values of x_i this formula is TRUE.

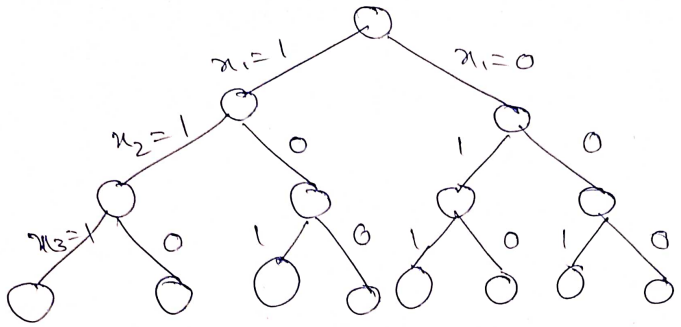
Possible values of x_i

x_1	x_2	x_3
0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1
1	1	0
1	1	1

time taken $2^3 \Rightarrow 2^n$ for n variables

This is also Exponential time problem.

State Space Tree : Soln to the problem can be represented as



We have to show that all exponential time algos are similar to satisfiability such that if satisfiability is solved in polynomial time all can be solved in polynomial time.

Example of 0/1 knapsack Problem

$P = \{10, 8, 12\}$ $n = 3$

$w = \{5, 4, 3\}$ $m = 8$ (capacity)

$x_i = \{0/1, 0/1, 0/1\}$

Satisfiability is NP-Hard.

NP-Hard : we are saying all Exponential time problems are hard problems. We solve them using Reduction. we show that satisfiability reducing to 0/1 knapsack. we take one formula & convert satisfiability into knapsack problem

NP-Hard NP-Hard

Sat $\propto$ 0/1 knapsack.

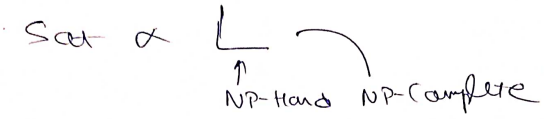
Anyone is solved all others will automatically be solved.

Sat $\propto L_1$ $L_1 \propto L_2$

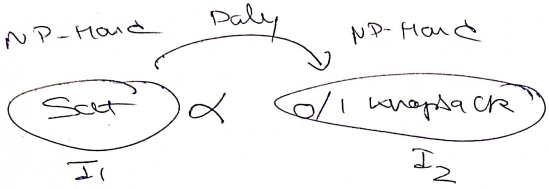
Sat $\propto$ L

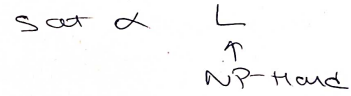
NP Hard

for Satisfiability we have Non deterministic polynomial algo so. it is NP. So satisfiability is NP Hard as well as NP-complete.



Satisfiability is NP-Hard. If satisfiability is reducing to 0/1 knapsack problem then this also becomes NP Hard



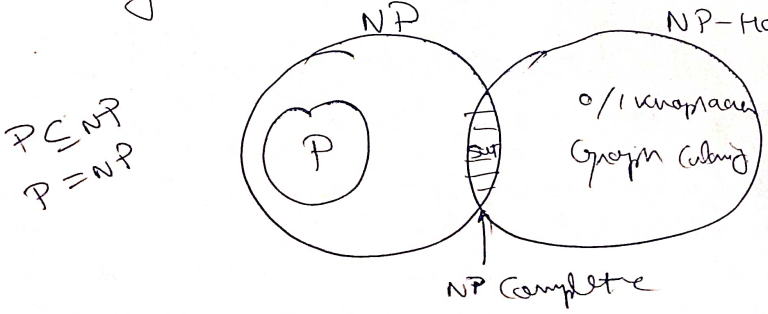


Sat αL_1 $L_1 \alpha L_2$

If L_1 is reducing to L_2 then L_2 also becomes NP Hard so any problem under exponential time is NP-Hard.

Satisfiability is NP Hard as well as NP-Complete

If a problem is having Non-deterministic time algo then that problem becomes NP-Complete.



P: Polytime time solvable problems

P ≠ NP both are decision problems {0,1}

All the problems solvable in polynomial time is $O(n^k)$

NP: Non Deterministic Polynomial time
↳ output is yes or no.

- NP problems are hard to solve but can be easily verifiable in Polynomial time.

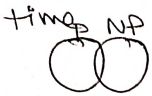
For Eg: we have a sudoku problem solve in front of you then you can solve it easily in polynomial time but if I give a sudoku problem unsolved then it will take exponential time. So we can say that all the problems that are solvable in exponential time but which are not solvable in polynomial time but can be verifiable in polynomial time are known to be Non deterministic Polynomial time Problems

Eg: Problem X → [A] (Algo A) → {0,1} gives decision
↳ A is in polynomial time
// P Problem

Eg: [Problem, {0,1}, Certificate] → [A] → yes / NO
// NP Problem verifiable in Polynomial time

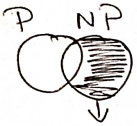
NP problems are those problems whose solution does not exist presently, but they can be verifiable in polynomial time.

If we can compute any problem in polynomial time so we can easily verify it easily in Polynomial



$P \subseteq NP$ // not complete subset

NP Hard: Any problem as hard as NP but not necessarily be in in NP. i.e it is atleast as hard as NP These are not even verifiable in P.



NP - P = NP Hard

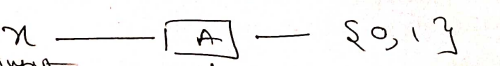
Reductions: If we have a problem A that is hard and we want to show that problem B is also hard

$A \Rightarrow B$ —— (1)

or we can say $B \Rightarrow A$ —— (2)

i.e B easy $\Rightarrow$ A easy

If we can show that if B is easy to solve in polynomial time then A is also easy to be solve in polynomial time, then we can say



we find $R: x \to f(x)$ s.t $A(x) = B[f(x)]$

$R: x \to y$

we know that B is easy and we can use this result to prove that A is also easy.

But since we know that A is hard ⇒ B must be hard there we used reducibility.

NP – Completeness Proof

If there is a problem X and we have to prove that it is NP – complete or not. There are 2 conditions:

(1) Problem belongs to NP i.e Problem has some Algo that should be solved in Non deterministic Polynomial time

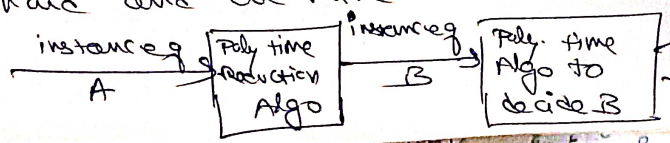
and there should be a verifier & certificate that the solution is correct. or not

$X \in NP$ → Non deterministic Algo
→ certificate + verifier

(2) Second find a problem y which is already NP complete and we have to reduce from know NP complete problem y to X.

Reduction: suppose there is a person B. and someone ask him could you touch moon from earth, so instead of saying no he replies if any human on this earth can touch moon then I can also touch. That is he has reduced the problem.

In reduction also, we have a problem A that is already hard and we have to show B is also hard.



How to Prove Vertex Cover Problem is NP Complete

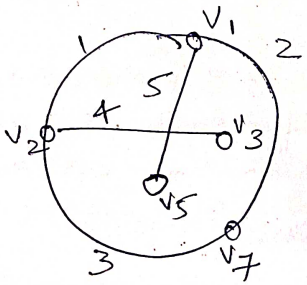
- **Vertex cover problem**

Vertex cover of unirected graph G(V,E) is subset $V' \subset V$ such that if $(u,v) \in G$ then

$u \in V'$
or
$v \in V'$
or
both u and $v \in V'$

Suppose we have a graph & the no. of vertices that include all the edges i.e all the edges are covered.

- Problem is to find minimum vertex that cover all the edges



"find minimum vertex that cover all edges in G"

If take vertex v_1 & v_2 then v_1 covers 1, 5, 2 edge and v_2 covers 1, 3, 4 edge i.e v_1 & v_2 are minimum vertices that covers all the edges

$V = [V_1, V_2, V_3, V_4, V_5]$

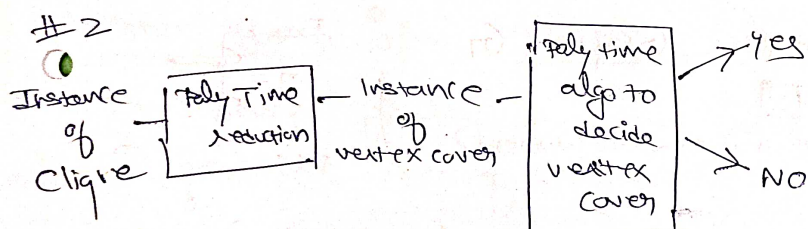
$V' = [V_1, V_2]$

Proof for NP Complete :

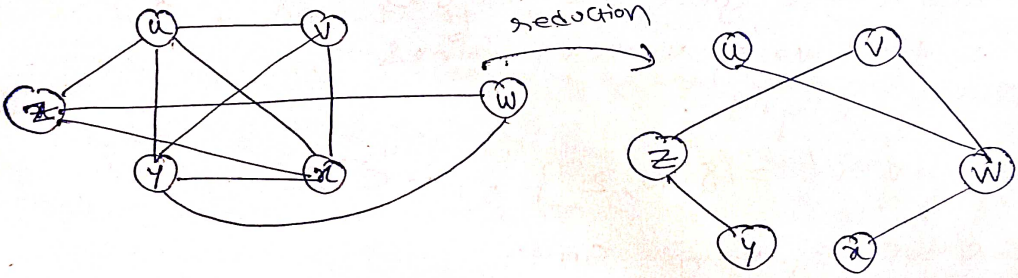
1. Vertex Cover Problem $\in$ NP. (verifier + certificate)
2. A NPC problem (Clique) can be reduced to Vertex cover Problem.

#1 There is a verifier that can give the certificate after checking that whether any vertex belong to all the edges or not. We can perform it in polynomial time i.e it is NP. If condition is true.

#2



3-CNF SAT (CNP Complete)
↓ reduced to
CLIQUE
↓ reduced to
Vertex Cover



3-CNF Sat problem was NP complete that was reduced to clique and clique is now going to reduce to vertex cover.

- we are given instances of clique are needed to be reduced to instances of vertex cover

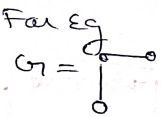
- we are given a graph in which value of clique (i.e max vertices those are connected to each other)
  i.e G(V,E) - clique $V' = \{u, v, x, y\}$
  ∴ value of clique = 4

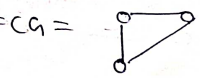
This graph is reduced so that we can get the instances of vertex cover i.e we find out

G' = fully complete Graph − G

i.e all vertices are connected

For eg





G' = FCG − G

=

G produced by reduction algo that has vertex cover is equal to the total no of vertex minus value of clique.

$V - V' = \{w, z\}$ i.e 2

∴ clique $\leq_p$ vertex cover

Vertex Cover Problem

- In the mathematical discipline of graph theory, "A vertex cover (sometimes node cover) of a graph is a subset of vertices which "covers" every edge.
- An edge is covered if one of its endpoint is chosen.
- In other words, "A vertex cover for a graph G is a set of vertices incident to every edge in G."
- The vertex cover problem: what is the minimum size vertex cover in G?

Vertex Cover is Greedy Algorithm

Idea: Keep finding a vertex which covers the the maximum no. of edges.

Step 1 : Find a vertex v with maximum degree.

Step 2 : Add v to the solution & remove v and all its incident edges from the graph.

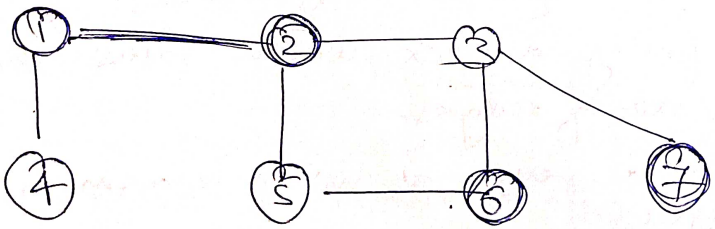
Step 3 : Repeat until all the edges are covered.

- Vertex - Cover problem is NP-Complete.
- A 2-approximation polynomial time algorithm is as the following:

```
APPROX-VERTEX-COVER (G)
C = ϕ;                          // C is a set of pair consisting of graph, ϕ is a null set
E' = G.E;
While (E' ≠ ϕ)
{
Randomly Choose a edge (u,v) in E', put u and v into C;
Remove all the edges that covered by u or v from E'
}
Return C;
```

Vertex-Cover Problem

Given a unirected graph, find a vertex Cover with minimum size.



Chose vertex 1, 5, 3

Vertex Cover — 3

final answer

Chose vertex: 1, 2, 6, 7

Vertex Cover — 4

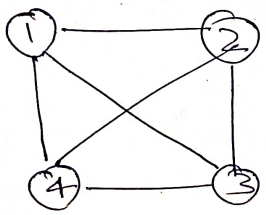
- A set of vertices such that each edge of the graph is incident to at least one vertex of the set, is called the vertex cover.

- Greedy algo may or may not produce optimal soln

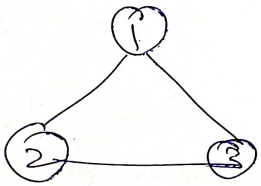
- Approximation algo does not always guarantee optimal solution but its aim is to produce a solution which is as close as possible to the optimal solution.

Clique Decision Problem (CDP)

Clique : From every vertex if there is an edge connecting to all other vertices then graph is complete graph.



(a)



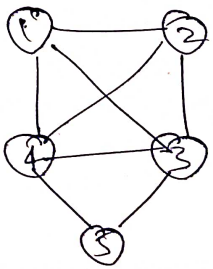
(b)

(c)

$|V| = n$ vertices

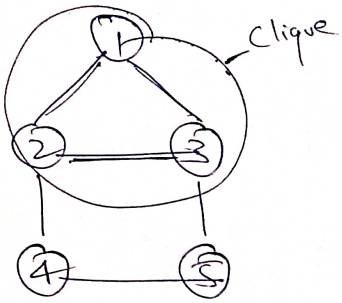
$|E| = \frac{n(n-1)}{2}$

Complete graphs



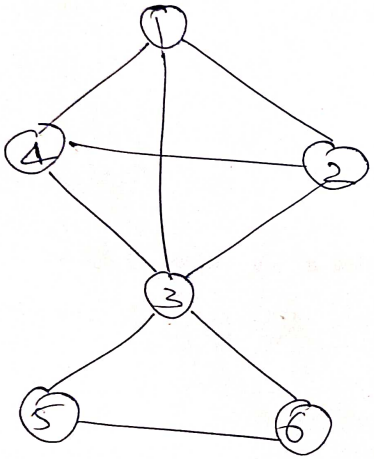
Clique is a subgraph of a graph which is complete.

Here if we add vertex (5) it is not complete, but its subgraph is complete.



(b)

Here we have added 2 more vertices ④ and ⑤. It is not complete. But subgraph of a graph is complete which is clique.



K = 4 (1, 2, 3, 4)

K = 3 (3, 5, 6)

K = 2 (5, 6)

Cliques

Q. is there a clique of size 2?
yes

Q. is there a clique of size 3 in this graph? yes

Decision Problem : A problem which requires answers as yes or no is a decision problem. i.e it is to find out true or false.

Optimization Problem : what is maximum Clique size in the graph ? 4. Finding the max or minimum is the optimization problem.

Clique Decision Problem is a graph problem & we have to prove that it is NP-Hard.

Suppose our problem is L_2 & we have to prove that L_2 is NP Hard. For proving we have to select one problem L_1 which is already NP Hard. From that we prove other problem is also NP Hard.

$P = L_2$

Satisfiability, $L_1 \propto L_2$ CDP

$I_1 \quad I_2$

If I_2 can be solved in polynomial time then I_1 can also be solved in polynomial time. Or if I_2 can be solved than I_1 can be solved.

We have to prove CDP as NP Hard which is L_2 & already known problem which is NP Hard is satisfiability